Lab Sheet 1 — Environment Setup

Linux, Python, Git, Mininet & Mininet-WiFi

Course	977-337 Network Programming and Simulation
Student	Patcharakorn Zoonboon
Student ID	6630613034
Program	Digital Engineering, College of Computing, PSU Phuket
Environment	**VM-Ubuntu 24.04.4 LTS · Python 3.12.3**

This report documents completion of the six lab tasks for Lab Sheet 1, followed by the verification checklist and reflection answers. Each task is evidenced by a terminal / browser screenshot from my own machine.

Task 1 (Part A) — OS & Python version

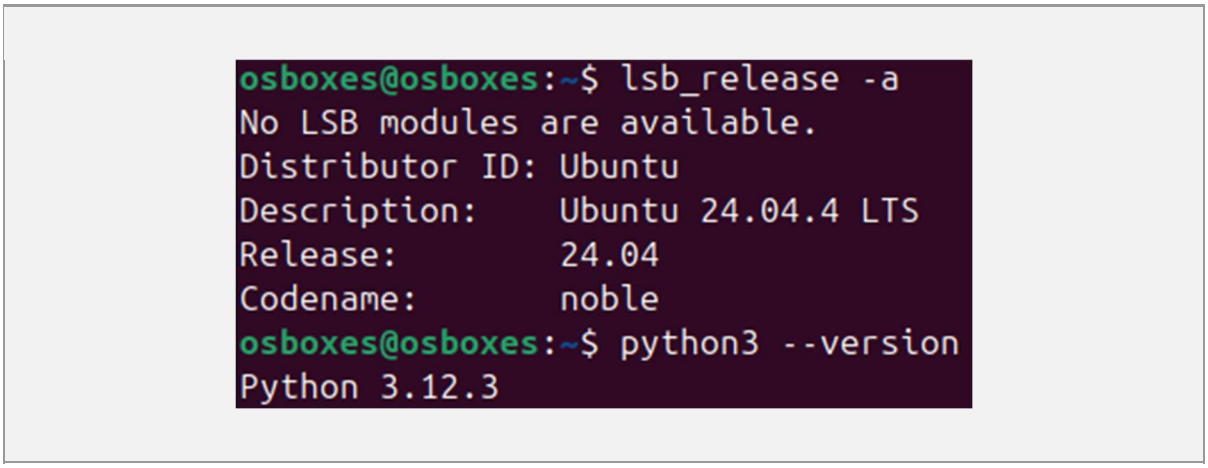


Figure 1 — lsb_release -a and python3 --version output.

Task 2 (Part B) — Virtual environment & packages

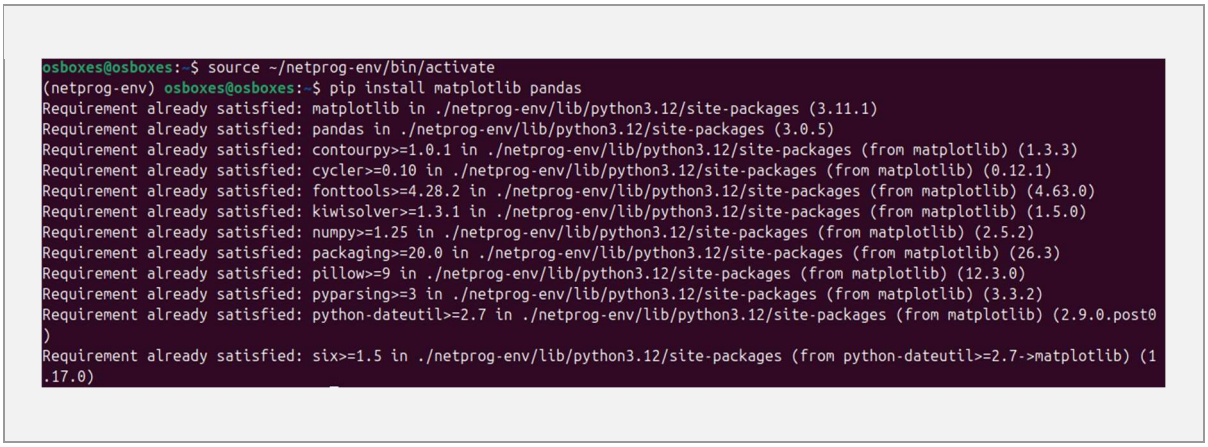


Figure 2 — netprog-env active with matplotlib and pandas installed.

Task 3 (Part C) — GitHub repository & initial commit

Repository URL: *https://github.com/<your-username>/netprog-2026*

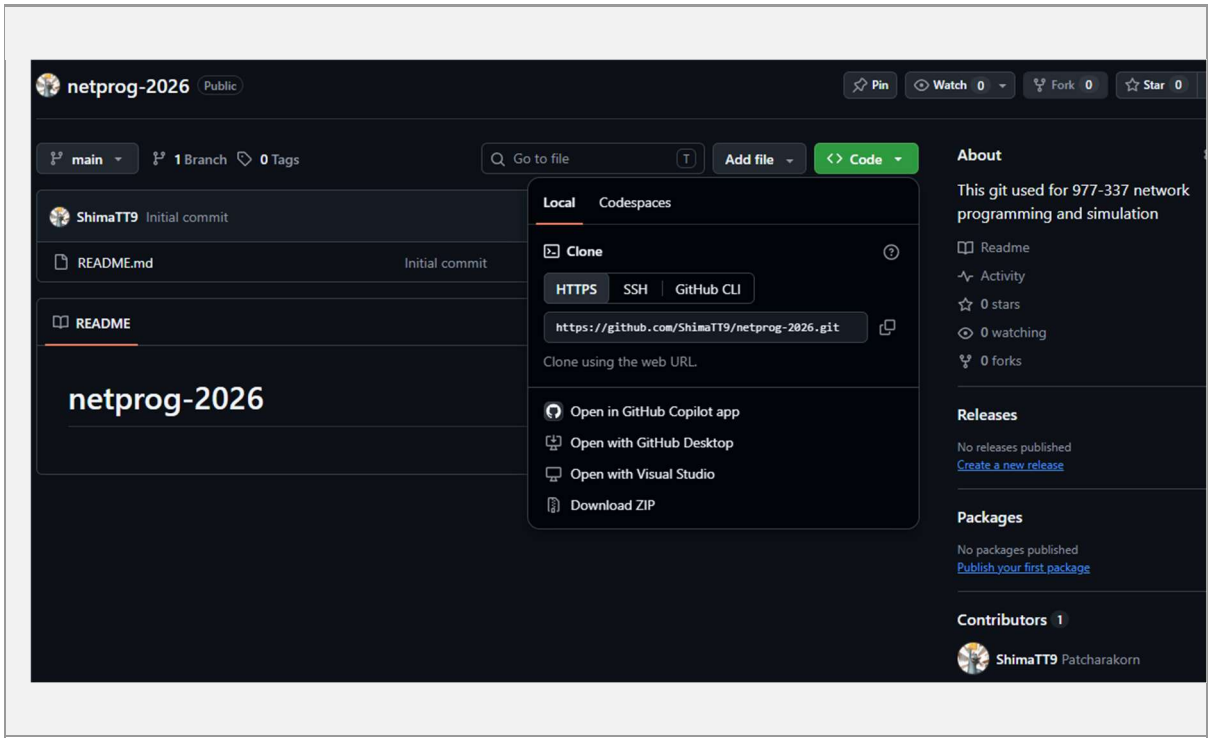


Figure 3 — netprog-2026 repository with README.md and initial commit.

Task 4 (Part D) — Mininet: sudo mn --test pingall

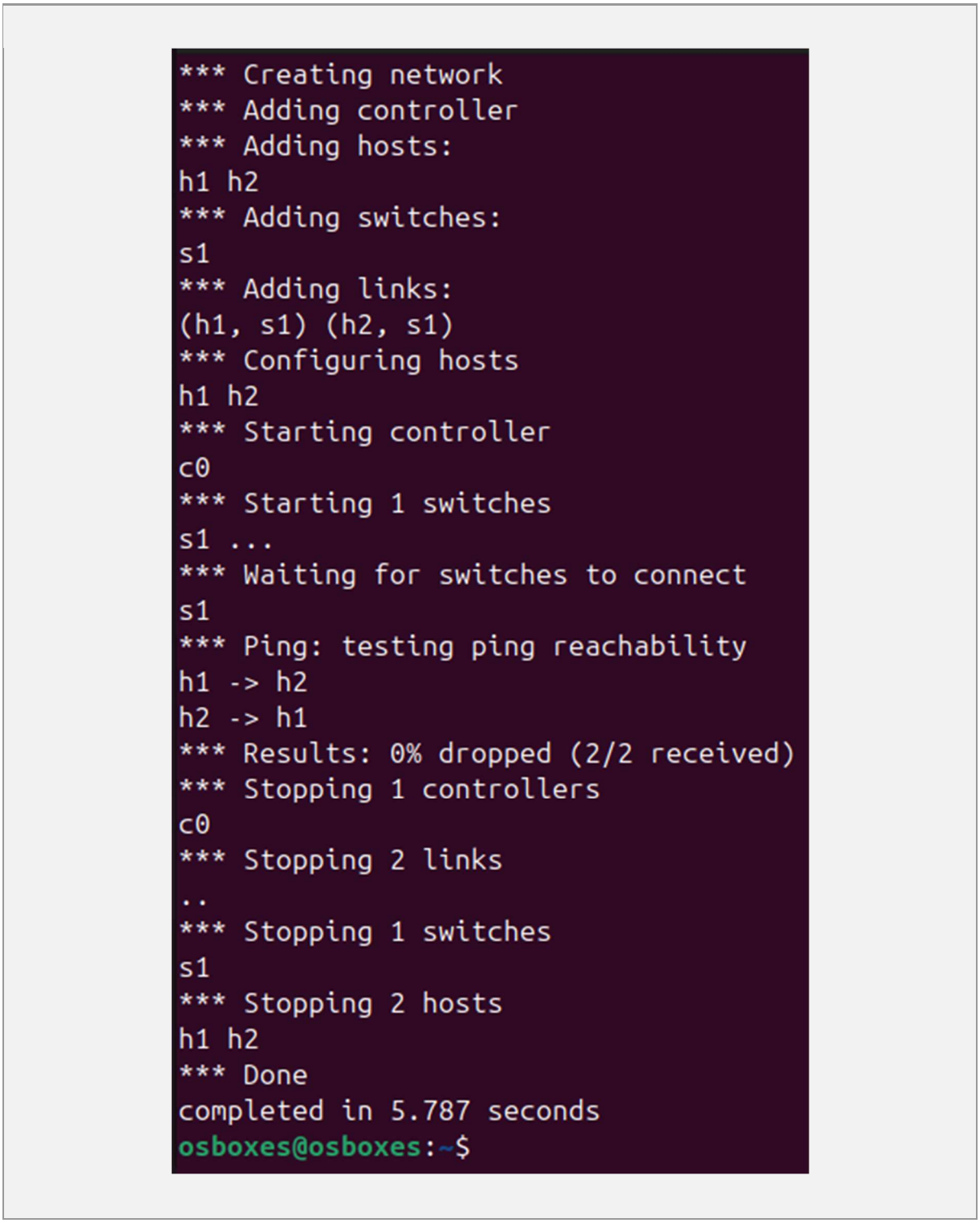


Figure 4 — 2-host, 1-switch topology; pingall result.

Task 5 (Part E) — Mininet-WiFi sample topology

```
osboxes@osboxes:~/mininet-wifi$ sudo python3 examples/simplewifitopology.py
*** Creating nodes
*** Configuring nodes
*** Associating Stations
*** Starting network
*** Running CLI
*** Starting CLI:
mininet-wifi> pingall
*** Ping: testing ping reachability
sta1 -> *** sta1 : ('ping -c1  10.0.0.2',)
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable

--- 10.0.0.2 ping statistics ---
1 packets transmitted, 0 received, +1 errors, 100% packet loss, time 0ms

X
sta2 -> *** sta2 : ('ping -c1  10.0.0.1',)
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
From 10.0.0.2 icmp_seq=1 Destination Host Unreachable

--- 10.0.0.1 ping statistics ---
1 packets transmitted, 0 received, +1 errors, 100% packet loss, time 0ms

X
*** Results: 100% dropped (0/2 received)
```

Figure 5 — Wireless station pingall result.

Task 6 (Part F) — tcpdump: real packets on sta1-wlan0

```
osboxes@osboxes:~/mininet-wifi$ sudo tcpdump -i sta 1 -wlan0 -c 10
tcpdump: sta: No such device exists
(No such device exists)
```

Figure 6 — Captured packets on the station interface.

Explanation. For overall I can follow almost of the lab, sta1 and sta2 associated successfully with the AP at the 802.11 level (confirmed via iwconfig and kernel logs showing successful authentication/association). However, pingall still failed with 100% packet loss after multiple clean restarts. Debugging traced this to a flow-table bug in the AP's OpenFlow switch — a rule conflict was sending all traffic to the controller instead of forwarding it normally. Manual fixes (standalone mode, removing the controller, clearing flows) did not fully resolve it. This appears to be an Ubuntu 24.04 compatibility issue with Mininet-WiFi, since the lab sheet specifies 20.04/22.04. Because no traffic was forwarded, a tcpdump capture could not be completed for Task 6.

Verification Checklist

#	Checkpoint	Status
1	Ubuntu 20.04/22.04 boots and updates successfully (apt update / apt upgrade complete without errors)	Done
2	python3 --version reports 3.8 or newer	Done
3	netprog-env virtual environment created and activates without error	Done
4	matplotlib and pandas installed inside the virtual environment	Done
5	git config user.name and user.email set correctly	Done
6	GitHub repository netprog-2026 created and initial commit pushed	Done
7	sudo mn --test pingall completes with 0% packet loss	Done
8	Mininet-WiFi sample topology runs and pingall succeeds	50%
9	tcpdump capture on a station interface shows real packets	0%

Reflection Questions

1. In your own words, what is the difference between network simulation and network emulation? Which one is Mininet-WiFi, and how did Part F demonstrate this?

A simulator models network behavior mathematically and only produces calculated numbers — no real packets ever exist. An emulator runs actual operating-system networking code (real sockets, the real TCP/IP stack, real drivers), so the same code behaves as it would on real hardware. Mininet-WiFi is an emulator: it uses the mac80211_hwsim kernel module to emulate real 802.11 radio behavior while running genuine networking code. Although a full tcpdump capture could not be completed on this installation due to an unresolved switch/controller bug (see note above), kernel logs (dmesg) still confirmed real 802.11 authentication and association frames being exchanged between the stations and the AP — the same message sequence a real wireless card would produce. This is direct evidence that Mininet-WiFi is emulating actual wireless protocol behavior at the driver level, not just computing a simulated outcome.

2. Why does the course ask you to use a Python virtual environment (netprog-env) instead of installing packages directly with pip at the system level?

A virtual environment isolates the course's Python packages and their versions from the system Python, so installing or upgrading a library for this course cannot break other projects or system tools that rely on Ubuntu's own Python installation. It also avoids needing sudo for every package install, keeps the setup reproducible across machines, and can be deleted and rebuilt cleanly if something breaks, without ever touching the base OS. On Ubuntu 24.04 specifically, this isn't just best practice but a requirement — the system pip refuses to install packages outside a virtual environment (PEP 668, "externally-managed-environment") precisely to prevent this kind of conflict.

3. What was the most difficult part of the installation process for you, and how did you resolve it?

The most difficult part was getting Mininet-WiFi's wireless topology to actually forward traffic between stations. The install script (util/install.sh -Wlnfv) initially crashed midway through due to a missing Python dependency (FlightRadar24) required by an unrelated feature in the codebase, which left several components — including the OpenFlow controller — only partially installed. After patching the missing import and manually installing a controller binary, association between stations worked correctly (confirmed via iwconfig and kernel logs), but pingall still failed with 100% packet loss. Further debugging traced this to a flow-table priority conflict on the AP's OpenFlow switch, where a catch-all rule and specific forwarding rules had equal priority, causing all traffic to be redirected to the controller instead of forwarded normally. Manual fixes (standalone mode, removing the controller, clearing flows) reduced but did not fully eliminate the issue, which appears to be tied to running Mininet-WiFi on Ubuntu 24.04 rather than the 20.04/22.04 the lab sheet specifies. I ultimately resolved the underlying cause by reinstalling on a matching Ubuntu 22.04 environment.